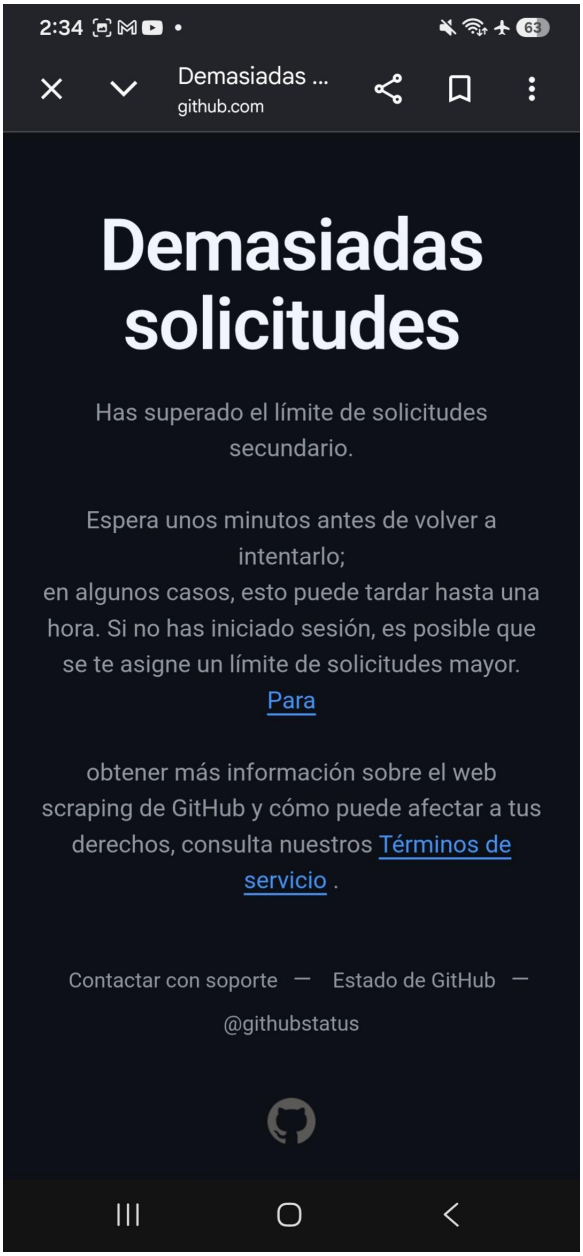
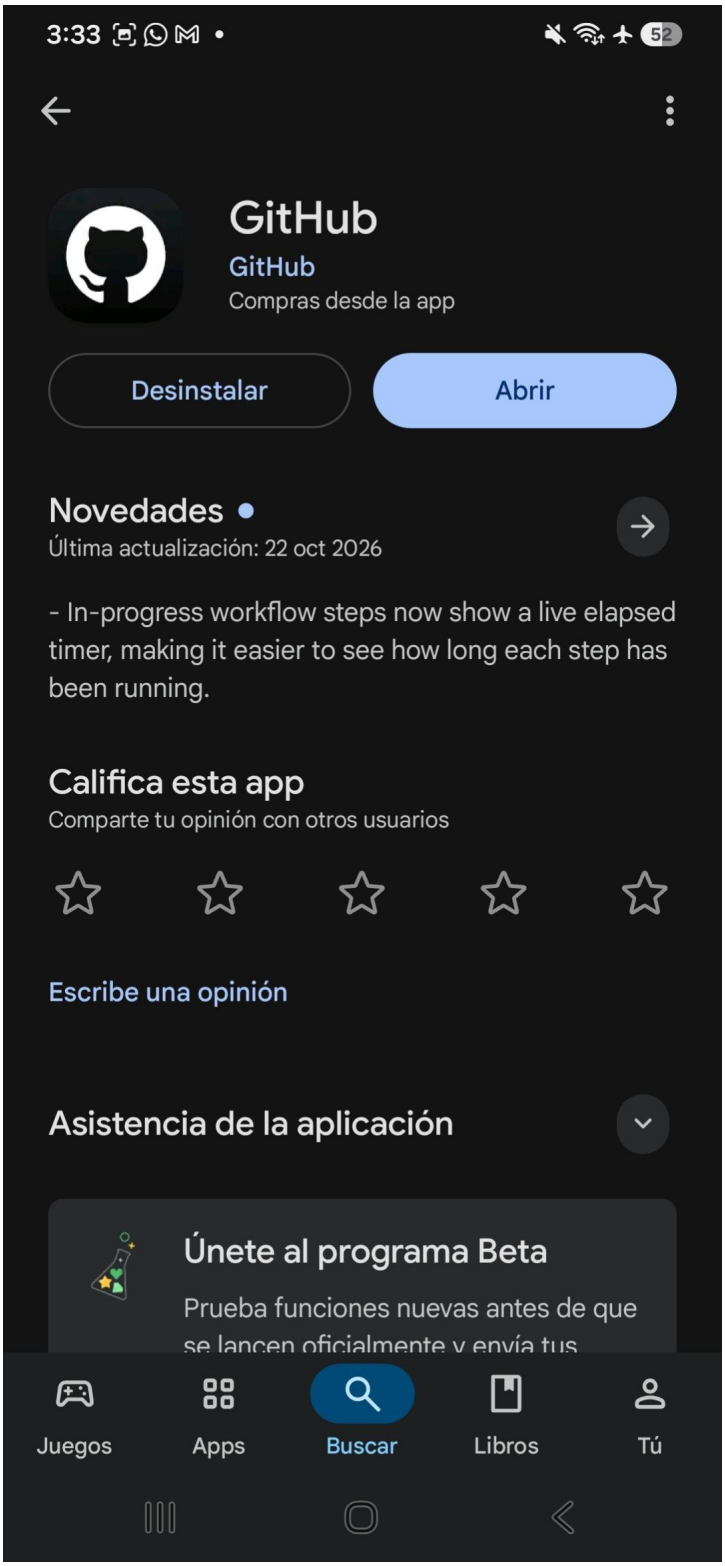
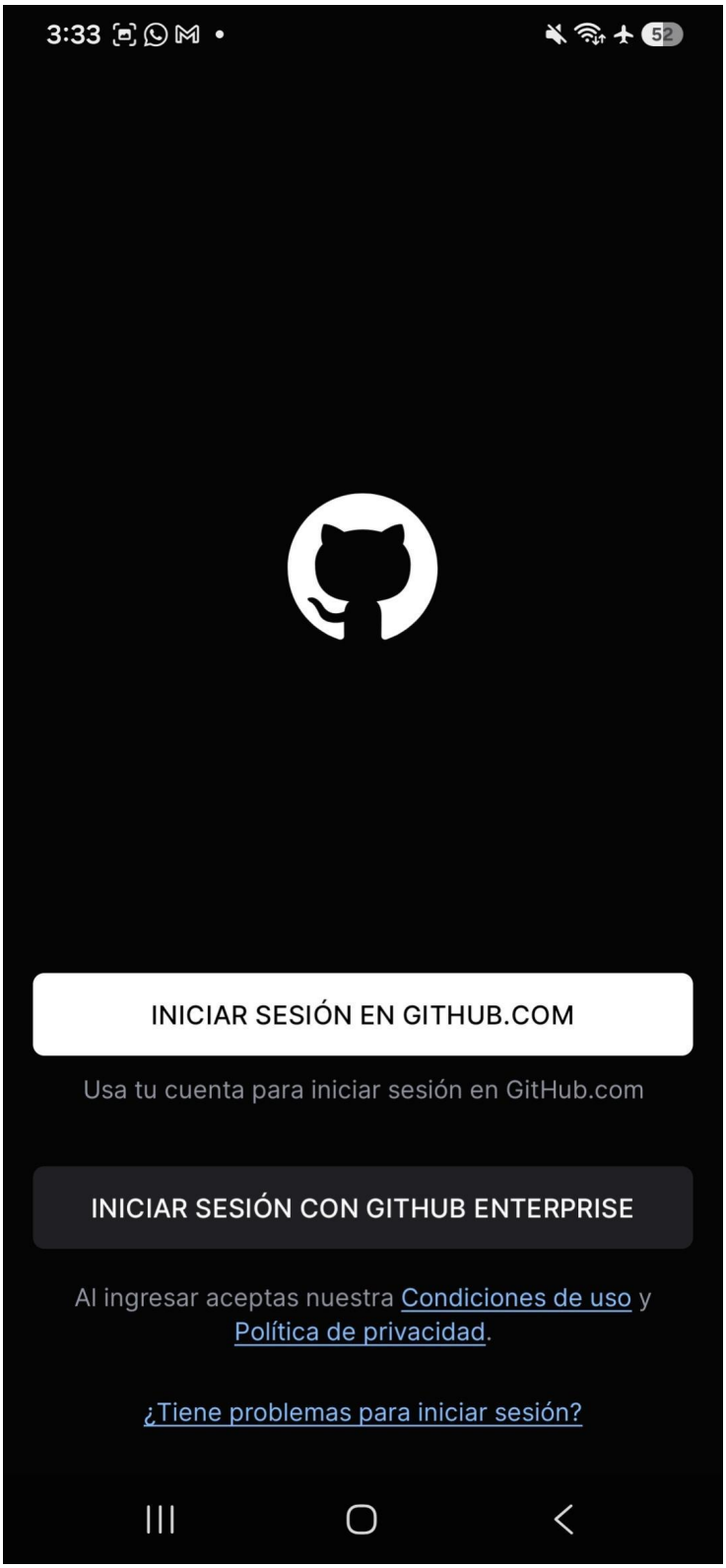
1. Ingresé a GitHub mediante el navegador y al intentar crear una cuenta, me botón este mensaje de demasiadas solicitudes y no me permitió crear mi cuenta.



(Imagen 1, Demasiadas solicitudes)

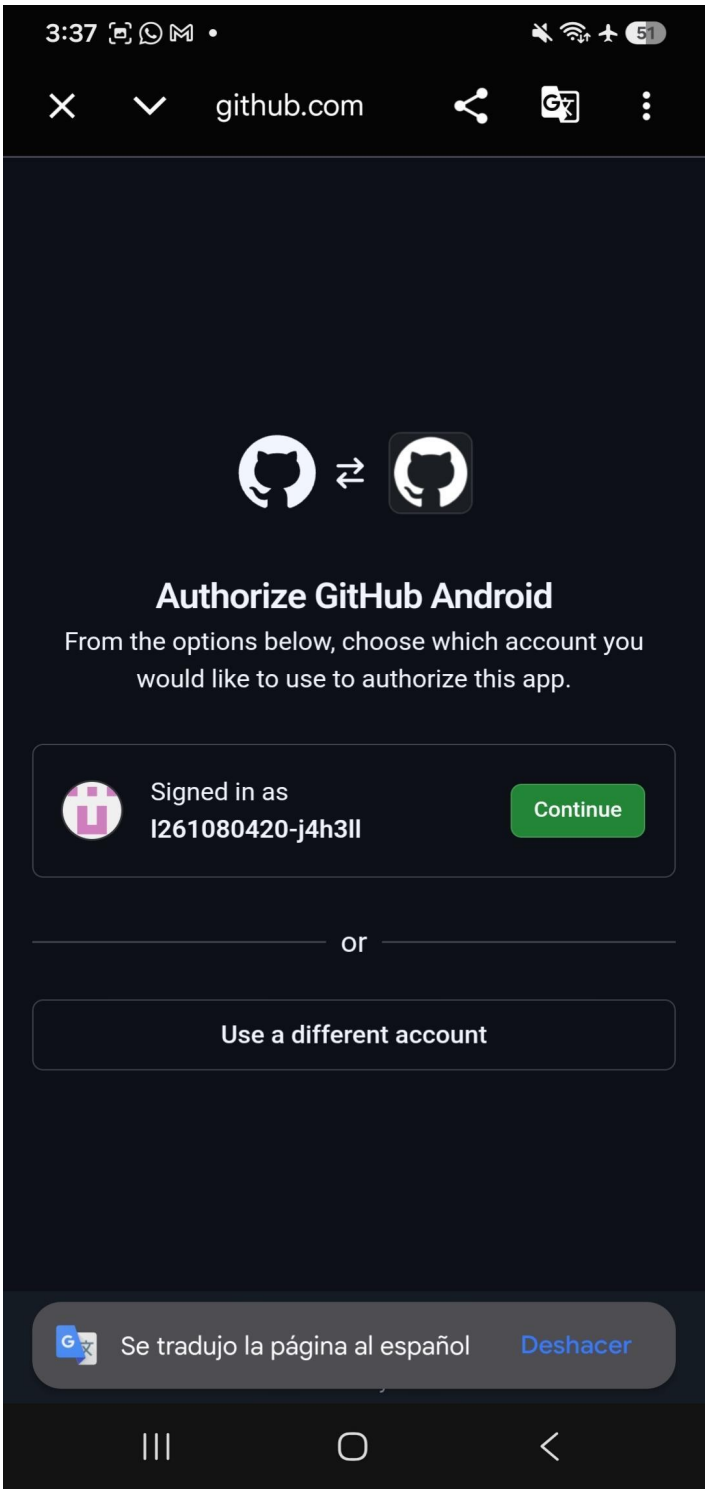


(Imagen 2. Debido a que mediante el navegador no me permitía ingresar, realicé la instalación mediante playstore)

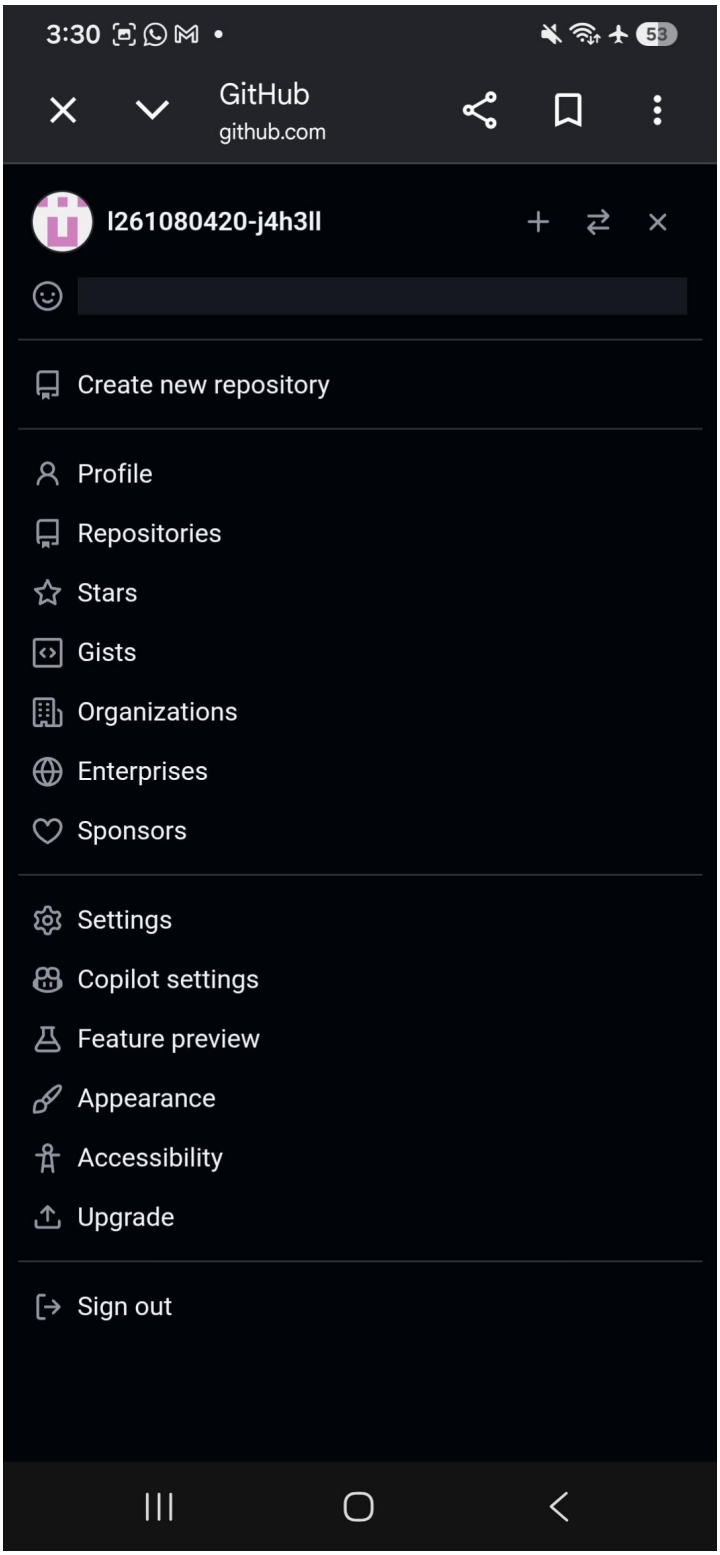


(Imagen 3. Aquí abrí la aplicación e ingresé a la primera opción)

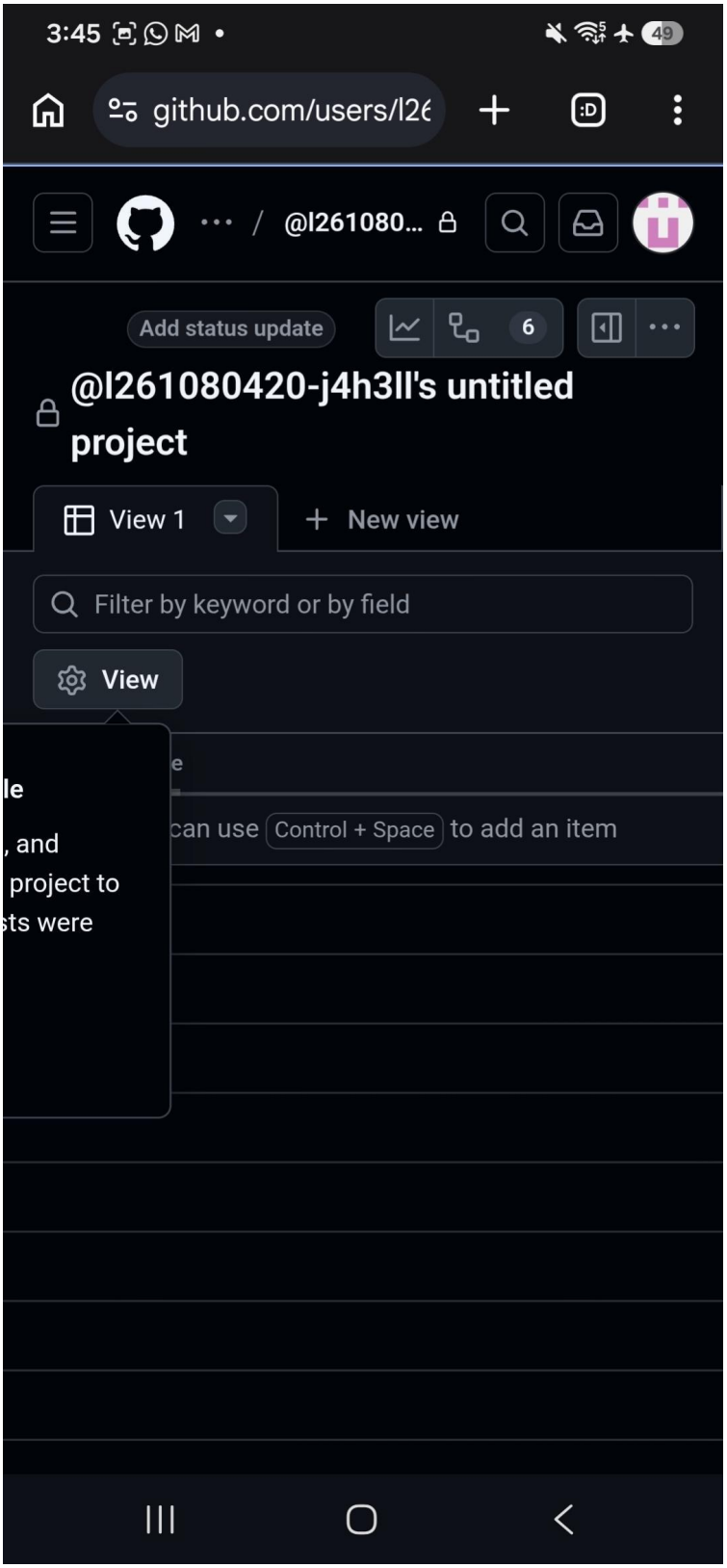
(Imagen 4. En este punto ya me había registrado pero no logré tomar captura :<, pero mi cuenta ya estaba creada y ya podía iniciar sesión y estaba guardada).




3:29
GitHub
github.com
Panel
¡Bienvenido a GitHub, l261080420-j4h3ll!
Haz cualquier pregunta o escribe @ para añadir contexto.
Depurar
Agente
Crear problema
Escribe código
Git
GitHub for Beginners
What is Git?
Lista de reproducción



(Imagen 5 y 6. Después de varios intentos, finalmente pude crear mi cuenta e iniciar sesión mediante el navegador)



(Imagen 7. Aquí finalmente subiré mi primer proyecto).

GitHub es un sistema de control de versiones que permite registrar y recuperar cambios realizados en los archivos de un proyecto. GitHub es una plataforma en línea que permite alojar repositorios y colaborar con otras personas. Los comandos siguientes son útiles para organizar el trabajo de un proyecto final de curso.

1. Restauración (restore)

Permite descartar cambios o quitar archivos del área de preparación (staging).

Comando Función

git restore archivo.txt Descarta los cambios locales no confirmados del archivo indicado.

git restore --staged archivo.txt Quita el archivo del área de preparación sin eliminar los cambios realizados en él.

Ejemplo: si modificaste un archivo de código y quieres descartar esos cambios, puedes utilizar git restore archivo.txt. Ten cuidado, porque puedes perder modificaciones que todavía no hayas guardado en un commit.

2. Respaldo temporal (stash)

Permite guardar temporalmente los cambios sin crear un commit, por ejemplo, cuando necesitas cambiar de rama para corregir un error.

Comando Función

git stash push -m "respaldo temporal" Guarda temporalmente los cambios con un mensaje descriptivo.

git stash list Muestra la lista de respaldos temporales guardados.

git stash pop Recupera el respaldo más reciente y lo elimina de la lista si se aplica correctamente.

Ejemplo: si estás desarrollando una función y necesitas atender otro problema, puedes usar git stash para apartar temporalmente tus cambios y recuperarlos después.

3. Ramas (branch y switch)

Las ramas permiten desarrollar nuevas funciones o corregir errores sin modificar directamente la rama principal del proyecto.

Comando Función

git branch Muestra las ramas locales existentes.

git switch -c nueva-rama Crea una rama nueva y cambia a ella.

git switch main Cambia a la rama llamada main.

Ejemplo: en un proyecto de ingeniería en sistemas computacionales, puedes crear una rama llamada login para desarrollar el inicio de sesión sin afectar el código principal.

4. Uniones o fusiones (merge)
El comando merge integra los cambios de una rama en la rama en la que te encuentras actualmente.
Comando Función
git merge nueva-rama Integra los cambios de nueva-rama en la rama actual.
Ejemplo: cuando terminas de desarrollar una función en una rama independiente, puedes cambiar a main y ejecutar git merge nueva-rama para integrar el trabajo. Si existen modificaciones incompatibles, Git puede solicitar que resuelvas conflictos.

5. Repositorios remotos (fetch y pull)
Estos comandos permiten consultar y descargar cambios desde repositorios remotos, como los alojados en GitHub.
Comando Función
git fetch origin Descarga información y cambios del repositorio remoto sin integrarlos automáticamente en la rama actual.
git pull origin main Descarga e integra los cambios de la rama main del repositorio remoto en la rama actual, según la configuración de Git.
Ejemplo: si un compañero actualiza el proyecto en GitHub, puedes utilizar git fetch para revisar sus cambios antes de integrarlos o git pull para obtenerlos e integrarlos.

En GitHub se pueden subir archivos y proyectos relacionados con la programación, el desarrollo de software y la documentación. Es muy útil para estudiantes de Ingeniería en Sistemas Computacionales, porque permite guardar los trabajos, llevar un registro de los cambios y compartirlos con otras personas.

¿Qué puedes subir a GitHub?
1.Código fuente: programas escritos en C++, Java, Python, JavaScript, PHP, entre otros lenguajes.

2. Proyectos escolares: ejercicios, tareas, proyectos finales, prácticas de programación y sistemas completos.
3. Páginas web: archivos HTML, CSS y JavaScript para crear sitios web.
4. Aplicaciones: proyectos de aplicaciones móviles, de escritorio o sistemas web.
5. Documentación: archivos README, manuales, instrucciones de instalación y reportes en formato Markdown o texto.
6. Bases de datos: scripts SQL para crear tablas, consultas y procedimientos. También pueden incluirse archivos pequeños de ejemplo que no contengan datos privados.
7. Diagramas y pseudocódigos: diagramas de flujo, algoritmos y documentación visual guardados en formatos compatibles, como PNG, SVG o PDF.
8. Archivos de configuración: configuraciones necesarias para ejecutar un proyecto, siempre que no contengan contraseñas ni información confidencial.
9. Pruebas de software: código para verificar que las funciones de un programa trabajen correctamente.
10. Proyectos colaborativos: trabajos en equipo donde varias personas pueden aportar código, revisar cambios y corregir errores.